

Lab Report

Sensor & Actuator – Ultrasonic Sensor and Linear DC Motor

Vehicle Electronics & Vehicle Communication Networks – SS2026
Technische Hochschule Ingolstadt

Participants:

- 1. Muhammad Arhum Siddiqui mus8857**
- 2. Waleed wam1633**
- 3. Zarrar Feroze Khan zak1232**
- 4. Muhammad Ahmed Khan muk9044**
- 5. Aftab Nabi Shaikh Afs7221**

Group-Number: 5
Date of Experiment: 11th May 2026
Date of Submission: 17th May 2026

<i>Experiment</i>	<i>Date of Experiment</i>	<i>Date of Submission</i>
<i>ACC</i>	<i>20 April 2026</i>	<i>26 April 2026</i>
<i>CISS</i>	<i>23 April 2026</i>	<i>29th April 2026</i>
<i>HIL</i>	<i>9th April 2026</i>	<i>14th April 2026</i>
<i>RADAR</i>		
<i>SENSOR&ACTUATOR</i>	<i>11th May 2026</i>	<i>17th May 2026</i>
<i>STEADY CIRCLE</i>	<i>16th April 2026</i>	<i>24th April 2026</i>

1. Introduction

This lab had two main parts. In the first part we tested an ultrasonic distance sensor (URM37) to understand how accurately it measures distance and what range of angles it can detect objects within. In the second part we used the sensor to control a linear DC motor actuator that simulates an automatic car boot opening system. The idea behind the boot system is the same as modern hands-free boot opening on cars: a sensor detects an object (like a foot or a hand) near the rear of the car and triggers a linear actuator to open or close the boot lid.

2. Background / Theory

2.1 Ultrasonic Sensor

An ultrasonic sensor works by sending out short pulses of sound at a frequency above the range of human hearing (typically around 40 kHz). These pulses travel through the air, hit an object, and bounce back to the sensor. The sensor measures the time it takes for the pulse to return – this is called the time-of-flight method. Because the speed of sound in air is known (343 m/s at 20°C), the distance to the object can be calculated as:

$$d = (t_{\text{roundtrip}} \times v_{\text{sound}}) / 2$$

The sensor used in this lab was the DFRobot URM37. It is a piezoelectric transducer, which means it can convert an electrical signal into a mechanical sound wave and also convert an incoming sound wave back into an electrical signal. The sensor operates in a monostatic configuration, meaning the same element is used both to send and receive the signal.

Analog and Digital Pins

The URM37 sensor has both analog and digital output modes. The digital pin (PWM output) sends a pulse whose duration is proportional to the measured distance. The Arduino reads this pulse using the `pulseIn()` function to calculate the distance. The analog pin outputs a voltage that changes with distance, which the Arduino can read with `analogRead()`. In our code we used the digital PWM output (pin 3 for echo, pin 10 for trigger) because it gives a direct time-of-flight reading without needing a voltage-to-distance conversion formula, making it simpler and more reliable at longer distances.

2.2 DC Linear Actuator

A DC linear actuator converts rotational motor motion into straight-line (linear) movement. Inside the actuator there is a small DC motor connected to a gearbox. The gearbox output drives a lead screw. As the lead screw rotates, a nut travels along it and pushes the sliding tube in or out. The direction of movement is reversed by swapping the polarity of the supply voltage.

In this experiment the motor used was the DSZY30-12V-A6-100mm-IP54 with a 16:1 gear ratio, 12V supply, and a maximum stroke of 100 mm. The full stroke requires 16.5 revolutions of the lead screw.

The H-Bridge module (IBT2 / BTS7960) was used to control the direction and speed of the motor from the Arduino. By setting RPWM high and LPWM low the motor extends; reversing these signals retracts it. The R_EN and L_EN pins were pulled HIGH to enable the bridge.

3. Experiment Execution

3.1 Ultrasonic Sensor – Distance and Angle Measurement

The sensor was mounted in a fixed position on the demo car. A round test object (40 cm diameter) made of wood and aluminium foil was placed at various distances in front of the sensor measured with a tape measure. At each position we read the distance reported by the sensor via the Arduino serial monitor. We also moved the object sideways and up/down to find the largest angle at which the sensor could still detect it reliably – this gives the horizontal and vertical field of view (FOV).

At distances beyond 400 cm the sensor began reporting invalid or wildly incorrect readings (for example 818 cm at a tape distance of 450 cm), which indicates the signal was too weak to be reliably reflected back. The usable maximum range was therefore taken as 400 cm.

Distance Measurement Results

Tape Distance (cm)	Sensor Reading (cm)	Error (cm)	Horiz. FOV (°)	Vert. FOV (°)
50	47	-3	40	35
100	100	0	40	40
150	147	-3	40	40
200	197	-3	40	30
250	247	-3	40	40
300	299	-1	—	—
350	351	+1	—	—
400	401	+1	—	—
450	Invalid / 818 (overshoot)	—	—	—
500	817 (~418)	—	—	—

Error Analysis – MAE and RMSE

To quantify how accurate the sensor is within its usable range (50–400 cm), we calculated two standard error metrics using the eight valid measurement points:

- Mean Absolute Error (MAE): average of the absolute differences between the tape distance and sensor reading.
- Root Mean Square Error (RMSE): emphasises larger errors more than MAE because individual errors are squared before averaging.

Calculations (using the 8 valid readings from 50 cm to 400 cm):

Tape (cm)	Sensor (cm)	Error (cm)	Abs Error (cm)	Error ² (cm ²)
50	47	-3	3	9

100	100	0	0	0
150	147	-3	3	9
200	197	-3	3	9
250	247	-3	3	9
300	299	-1	1	1
350	351	1	1	1
400	401	1	1	1

$$MAE = (3 + 0 + 3 + 3 + 3 + 1 + 1 + 1) / 8 = 1.88 \text{ cm}$$

$$RMSE = \sqrt{(9 + 0 + 9 + 9 + 9 + 1 + 1 + 1) / 8} = 2.21 \text{ cm}$$

The MAE of 1.88 cm and RMSE of 2.21 cm show that within its usable range the sensor is very accurate. The datasheet specifies 1% accuracy; at 400 cm that would be 4 cm, so our results are consistent with the specification. The slightly larger errors at shorter distances (50–250 cm) are likely due to small alignment differences between the sensor axis and the tape measure line.

Field of View (FOV) and Maximum Range

The field of view was measured by moving the object away from the central axis (horizontally and vertically) until it was no longer detected. The last angle at which detection was still reliable was recorded as the FOV boundary.

Distance (cm)	Horizontal FOV (°)	Vertical FOV (°)
50	40	35
100	40	40
150	40	40
200	40	30
250	40	40

From these results we can derive the following:

- Horizontal FOV: consistently 40° across all measured distances, giving a reliable beam cone of approximately $\pm 20^\circ$ from the sensor axis in the horizontal plane.
- Vertical FOV: 35°–40° depending on distance, averaging around 38°. The slightly narrower angle at 200 cm is likely a measurement artefact.
- Maximum usable range: 400 cm. At 450 cm the sensor produced invalid readings and at 500 cm it reported a wildly wrong value (817 cm). This matches the datasheet stated range of 800 cm ultimate range, but in practice reliable readings stopped around 400 cm, probably because the test object's surface was not perfectly reflective.

3.2 Controlling the Linear Actuator with the Ultrasonic Sensor

Once the sensor was tested, we connected it to the linear actuator through the IBT2 H-Bridge. The complete system was designed to simulate an automatic car boot: when an object is detected within 30 cm of the sensor, the actuator toggles between the extended (boot open) and retracted (boot closed) positions.

The Arduino code works as follows:

- The sensor triggers a measurement every 200 ms by pulling the TRIG pin briefly LOW then HIGH.
- The echo returns as a LOW pulse on the echo pin. The pulse width in microseconds divided by 50 gives the distance in cm.
- If the distance is between 1 cm and 30 cm, a toggle flag (check) switches state.
- When check = 0: RPWM is set to 255 and LPWM to 0, driving the motor outward (boot opens) for 10 seconds.
- When check = 1: RPWM is set to 0 and LPWM to 255, driving the motor inward (boot closes) for 10 seconds.

The 10-second delay gives the motor enough time to complete the full 100 mm stroke before the next detection event is processed. The built-in limit switches inside the actuator protect it from overdriving at either end.

4. Discussion

The ultrasonic sensor performed well within its rated range. The MAE of 1.75 cm and RMSE of 2.08 cm correspond to relative errors well below 2%, which is within the 1% accuracy specification stated in the datasheet (the datasheet tolerance applies to nominal conditions; at the short end of the range small absolute errors become larger relative errors).

The field of view was consistent at approximately 40° horizontal across all tested distances. This wide beam means the sensor is well-suited for detecting objects in a general area but not precise enough to locate them in a narrow beam – which is fine for a boot-opening trigger application where the foot just needs to be somewhere below the bumper.

The failure at 450–500 cm is expected for a wood/foil target in an indoor environment. Sound scatters at long distances and the reflected signal becomes too weak to be reliably detected above the noise floor. In a real car the bumper sensor would operate over much shorter distances (typically under 200 cm for parking sensors).

The motor control worked correctly. The toggle logic using the check variable means the system remembers whether the boot is open or closed and always moves it to the opposite position on the next trigger event. The 10-second motor run time is a fixed delay; in a real system one would instead use the limit switch feedback to stop the motor exactly when it reaches the end of its stroke.

5. Arduino Code

The following code was written for the Arduino Mega board to read distance from the URM37 ultrasonic sensor and use it to control the linear actuator via the IBT2 H-Bridge. The code is written in the Arduino dialect of C.

Pin Definitions and Global Variables

URECHO (pin 3) receives the echo pulse from the sensor. URTRIG (pin 10) sends the trigger signal to start a measurement. R_EN and L_EN (pins 6 and 7) enable the left and right sides of the H-Bridge. R_PWM and L_PWM (pins 4 and 5) set the direction and speed of the motor using PWM signals. The variable check acts as a toggle flag to remember whether the boot is currently open or closed.

Setup Function

The trigger pin is set HIGH to keep the sensor ready. The echo pin is set as an input. All four H-Bridge pins are set as outputs and both enable pins are pulled HIGH to activate the bridge. A short delay allows the sensor to initialise before measurements begin.

Loop Function – Distance Measurement

Each iteration of the loop pulls the trigger pin LOW then immediately HIGH. This brief LOW pulse tells the URM37 to fire an ultrasonic burst. The sensor then pulls the echo pin LOW for a duration proportional to the measured distance. The Arduino measures this LOW pulse using pulseIn(URECHO, LOW). If the pulse is 50,000 microseconds or longer the reading is considered out of range and reported as invalid. Otherwise the pulse duration is divided by 50 to convert to centimetres (the URM37 datasheet specifies that each 50 microseconds of LOW pulse corresponds to 1 cm of distance).

Loop Function – Motor Control

After the distance is measured, the code checks whether it falls in the trigger zone (1 to 30 cm). If so, the check flag is examined. If check is 0 the motor is driven outward by setting R_PWM to full (255) and L_PWM to 0; this extends the actuator and represents opening the boot. The flag is then set to 1. If check is already 1 the signals are reversed (R_PWM = 0, L_PWM = 255), retracting the actuator and closing the boot, and the flag is reset to 0. A 10-second delay after each trigger gives the motor time to complete its full 100 mm stroke before the next detection event is processed.

```
int URECHO = 3;           // Echo pin (PWM output from sensor)
int URTRIG = 10;          // Trigger pin
int R_EN   = 6;           // H-Bridge right enable
int L_EN   = 7;           // H-Bridge left enable
int R_PWM  = 4;           // Right PWM (forward)
int L_PWM  = 5;           // Left PWM (reverse)
int check  = 0;           // Toggle: 0 = boot closed, 1 = boot open
unsigned int DistanceMeasured = 0;

void setup() {
  Serial.begin(9600);
  pinMode(URTRIG, OUTPUT);
  digitalWrite(URTRIG, HIGH); // Keep HIGH to enable sensor
  pinMode(URECHO, INPUT);
  delay(500);
  pinMode(R_EN, OUTPUT); pinMode(L_EN, OUTPUT);
  pinMode(R_PWM, OUTPUT); pinMode(L_PWM, OUTPUT);
  digitalWrite(R_EN, HIGH);
  digitalWrite(L_EN, HIGH);
}

void loop() {
  digitalWrite(URTRIG, LOW);
  digitalWrite(URTRIG, HIGH); // Trigger measurement
```

```

unsigned long LowLevelTime = pulseIn(URECHO, LOW);

if (LowLevelTime >= 50000) {
    Serial.println("Invalid");
} else {
    DistanceMeasured = LowLevelTime / 50; // 50 us = 1 cm
    Serial.print(DistanceMeasured);
    Serial.println(" cm");
}
delay(200);

if (DistanceMeasured > 0 && DistanceMeasured < 30) {
    if (check == 0) {
        analogWrite(R_PWM, 255); // Extend (open boot)
        analogWrite(L_PWM, 0);
        delay(10000);
        check = 1;
    } else {
        analogWrite(R_PWM, 0); // Retract (close boot)
        analogWrite(L_PWM, 255);
        delay(10000);
        check = 0;
    }
}
}
}

```

6. Conclusion

This lab successfully demonstrated the use of an ultrasonic sensor and a linear DC actuator in an automotive-inspired application. The key findings are:

- The URM37 sensor is accurate to within approximately 1.75 cm (MAE) over a range of 50–400 cm.
- The horizontal field of view is approximately 40° and the vertical FOV is approximately 35–40°.
- The maximum reliable detection range with the test object was 400 cm; beyond this the sensor produced invalid or heavily incorrect readings.
- The boot-opening trigger logic worked as expected: a hand placed within 30 cm of the sensor alternately extended and retracted the linear actuator.
- Analog pins output a voltage proportional to distance; digital pins output a timed pulse. We used the digital (PWM) pin because it gives a direct time-of-flight measurement without a conversion formula.

7. References

DFRobot. URM37 V5.0 Ultrasonic Sensor Wiki.

https://wiki.dfrobot.com/URM37_V5.0_Ultrasonic_Sensor_SKU_SEN0001_

MSW Motion Control GmbH. DSZY30-12V-A6-100mm-IP54 Datasheet. www.msw-motion.de

Arduino. Language Reference. <https://www.arduino.cc/reference/en/>